

Written for: alsy7009's AI agent (context to paste into their own Claude/agent session so it has full project state before starting work).

PROJECT: Baton — HackMIT 2026, Healthcare Track

Repo: `leenadudi/baton` on GitHub. Issue tracker is the source of truth for task breakdown.

What Baton is: Baton drafts the nursing/resident shift handoff from a patient's chart and flags what's missing, contradicted, or stuck — before the next shift has to catch it by re-reading everything. Three flag types:

1. Conflicting instructions — different care team members left contradictory orders/notes on the same topic (diet, anticoagulation, weight-bearing, etc.). Baton shows both source snippets and authors; it does not decide who's right.
2. Incomplete handoffs — required fields missing (code status, allergies, follow-up owner, medication reconciliation).
3. Stuck administrative blockers — authorizations, referrals, equipment requests that haven't moved.

Baton does not diagnose or recommend treatment. This matters for the pitch — it's explicitly a coordination/operations tool, matching the healthcare track's own stated framing almost word for word.

Architecture — standard FHIR, no custom data model: This is a deliberate positioning choice against Epic and Oracle Health, which already ship handoff-activity forms clinicians retype into manually. Baton's pitch is "they give you the form, we fill it in and tell you what's missing," which only holds up if the data model is real FHIR, not invented.

Data	FHIR resource
Notes (free text)	<code>DocumentReference</code> , <code>DiagnosticReport</code>
Meds	<code>MedicationRequest</code>
Allergies	<code>AllergyIntolerance</code>
Code status / goals	<code>CarePlan</code> , <code>Goal</code> , <code>Consent</code>
Care team membership	<code>CareTeam</code> , <code>PractitionerRole</code>
Consults / referrals	<code>ServiceRequest</code>
Follow-up owner, admin blockers	<code>Task</code> (lean on this heavily — it already has owner/status/businessStatus/timestamps, so "unassigned follow-up" and "blocker stuck 4 days" become plain queries, not bespoke logic)

Free-text notes get extracted into structured `{role, topic, value}` tags via the OpenAI API. A separate, untouched rule engine (already written in JS in `frontend/public/prototype.html`, functions `findConflicts/getIssues`) decides what counts as a conflict — the model reads, the rules decide, every flag stays auditable to a source note. This targets the OpenAI Challenge sponsor prize.

Write-back is deliberately minimal: generate-and-copy only, zero write permissions to any FHIR server. No signed-note write-back, ever, in this build.

There is no FHIR event for "shift change" — that timing lives in separate scheduling systems (Amion, QGenda, Teams Shifts). For the hackathon demo this is faked with a timer; it's framed as a real finding for the pitch, not hidden as a gap.

Stack: React + Vite frontend, FastAPI backend, both currently minimal starters.

`frontend/public/prototype.html` is a working single-file reference UI with hardcoded demo data and the real rule-engine logic — treat it as the spec for both frontend and backend behavior, not just a mockup.

Team (4 people), by GitHub issue:

- Sophie (`sophieluo08-boop`) — frontend/design. Start: [#13](#)
- Leena (`leenadudi`, repo owner) — backend/API/FHIR client. Start: [#14](#)
- Nicole (`nmzheng`) — OpenAI extraction pipeline. Start: [#15](#)
- **alsy7009 (you)** — FHIR test data. Start: [#16](#)

Your task specifically: Generate synthetic-but-realistic patient records with Synthea, load them into the public HAPI FHIR test server (no approval/account needed for either), and confirm every resource type in the table above is actually queryable. Synthea's own generated notes tend to be thin/templated — if so, backfill richer note text from the MTSamples dataset (free, CC0, on Kaggle) so Nicole's extraction pipeline has real language to work with. Hand the live FHIR server URL to Leena once it's populated so she can build the backend's FHIR client against it. Full step-by-step detail is in [#16](#); the raw resource-mapping reference is [#7](#).

Constraints: 24-hour hackathon. Cut mobile polish, live-wiring all patients (2–3 demo patients is enough, rest as seed data), and any dataset requiring a data-use-agreement (MIMIC-IV, n2c2 — approval won't clear in time) before cutting the core FHIR/extraction path.

Want me to also add this as a pinned comment on #16, or is pasting it into their agent enough?